ARCHITECTURE GUIDE — FOR BEGINNERS

How Your App Is Built

No React knowledge needed. Diagrams, analogies, and plain English only.

> **What this guide does:** It shows you the skeleton of your app — how the pieces connect, why they exist, and what each one does. After reading this you will be able to explain your own architecture to your mentor with confidence.

1 The Simplest Possible Explanation

ANALOGY — THINK OF IT LIKE A RESTAURANT

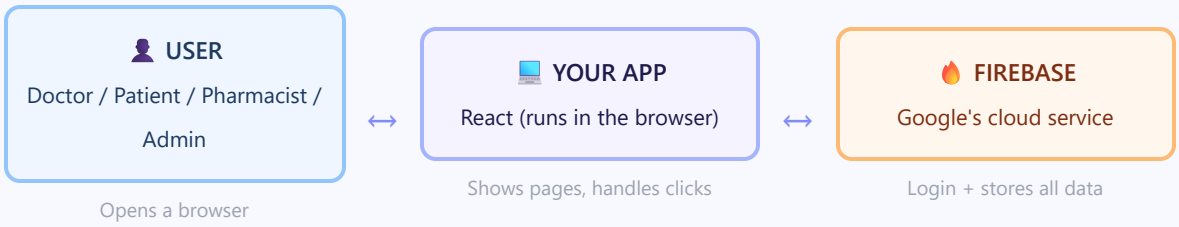
The customer (user) sits at a table and sees only the menu and their plate.

The waiter (your React app) takes the order, shows results, and talks to the kitchen.

The kitchen (Firebase) stores all the food (data) and processes orders.

The customer never goes into the kitchen. The kitchen doesn't talk to the customer directly. The waiter (your app) is always in the middle.

The Big Picture — 3 Players



Key point: Your app (React) lives in the browser — not on a server you built. Firebase is the only server involved, and Google runs it for you. You wrote zero backend code.

Say to your mentor: *"The app has three layers. The user interacts with a React interface that runs in the browser. When data needs to be saved or retrieved — like a prescription or a user's role — the app talks to Firebase, which is Google's cloud service. Firebase handles both login security and the database."*

2 The 3 Layers of Your App

Every web application has layers — like a building has floors. Your app has three:

LAYER 1

What the User Sees — The UI Layer

All the pages, buttons, forms, cards, and dashboards. This is built with **React** (the components) and styled with **Tailwind CSS** (the colours and layout). Think of this as the paint, furniture, and windows of a building.

↓

LAYER 2

The Logic Layer — How the App Thinks

The rules that govern behaviour. Who is logged in? What role do they have? Which page should they see? What happens when a form is submitted? This is handled by **React Context** (global state), **React Router** (navigation), and the **RequireRole guard** (access control). Think of this as the electrical wiring and plumbing inside the walls.

↓

LAYER 3

Where Data Lives — The Data Layer

All the real information: user accounts, prescriptions, medicines, inventory, orders. This is stored in **Firebase Firestore** (the database) and protected by **Firebase Authentication** (login system). Think of this as the basement vault where everything important is kept.

> **Say to your mentor:** *"The app is structured in three layers. The top layer is the UI — what the user sees, built with React and styled with Tailwind. The middle layer is the logic — routing, role checking, global state. The bottom layer is Firebase — authentication and the Firestore database where all data is stored."*

3 How the Code Is Organised — The Folder Structure

ANALOGY — THINK OF IT LIKE AN OFFICE BUILDING

Each role has its own floor (folder). The doctor's team works on Floor 2, the pharmacy team on Floor 3. Common facilities — the lobby, the elevators, the Wi-Fi — are in the **shared** area. No team needs to go to another team's floor to do their job.

```
📁 src/ ← All the source code lives here

  📁 app/
    main.jsx
← Starts the app (the "ON" switch)
    router.jsx
← The map: which URL shows which page
    providers.jsx
← Wraps everything so login info is available everywhere
  📁 auth/
    pages/
← Login page, Sign up page
  📁 context/
    AuthContext.jsx
← The shared notice board (who is logged in, what role)
  📁 guards/
    RequireRole.jsx
← The security guard that blocks wrong-role users
  📁 roles/ ← Each role has its own separate world
    📁 doctor/
      pages/
← New Prescription, Patients, Pharmacies…
      components/
← UI pieces only doctors see
    📁 patient/
```

```
      pages/
← My Doctors, Dose Schedule…
    📁 pharmacy/
      pages/
← Inventory, Medicines, Orders…
    📁 admin/
      pages/
← Users management
  📁 shared/ ← Used by ALL roles
    components/
← Sidebar, layout, buttons, cards (reused everywhere)
    pages/
← Search, Prescriptions list, Settings (all roles use these)
    lib/
← Firebase connection + database helper functions
```

> **Say to your mentor:** *"I used a role-based folder structure. Instead of having one big components folder, each role has its own isolated folder with its own pages and components. Code shared by all roles — like the sidebar or the prescriptions list — lives in a shared folder. This makes it easy to find things and easy to add a new role without touching existing code."*

4 The Role System — Who Sees What

ANALOGY — THINK OF IT LIKE A HOTEL KEY CARD

When you check into a hotel, your room key only opens your room and shared areas (gym, pool). It does not open other guests' rooms or the staff office. Your role in MedFind works the same way — it unlocks only the pages that belong to you.

WHO GETS WHAT

Doctor	Dashboard · New Prescription · Patient List · Pharmacy List · Prescriptions · Connections · Search · Settings
Patient	Dashboard · My Prescriptions · Dose Schedule · My Doctors · Search · Settings
Pharmacist	Dashboard · Inventory · Medicines · Orders · Prescriptions · Search · Settings
Admin	Dashboard · Users · Doctors · Patients · Pharmacies · Prescriptions · Search · Settings

How does the app know which role the user has?

1. **User signs up and picks a role**
   When creating an account, they choose: Doctor, Patient, or Pharmacist. This role is saved in Firebase.

2. **On login, the app fetches the profile**
   Firebase confirms the password. Then the app goes to Firestore and gets the user's profile — which includes their role.

3. **Role is stored in the global notice board (Context)**
   The role is placed in React Context — a shared memory space. Now every part of the app can read "this user is a Doctor."

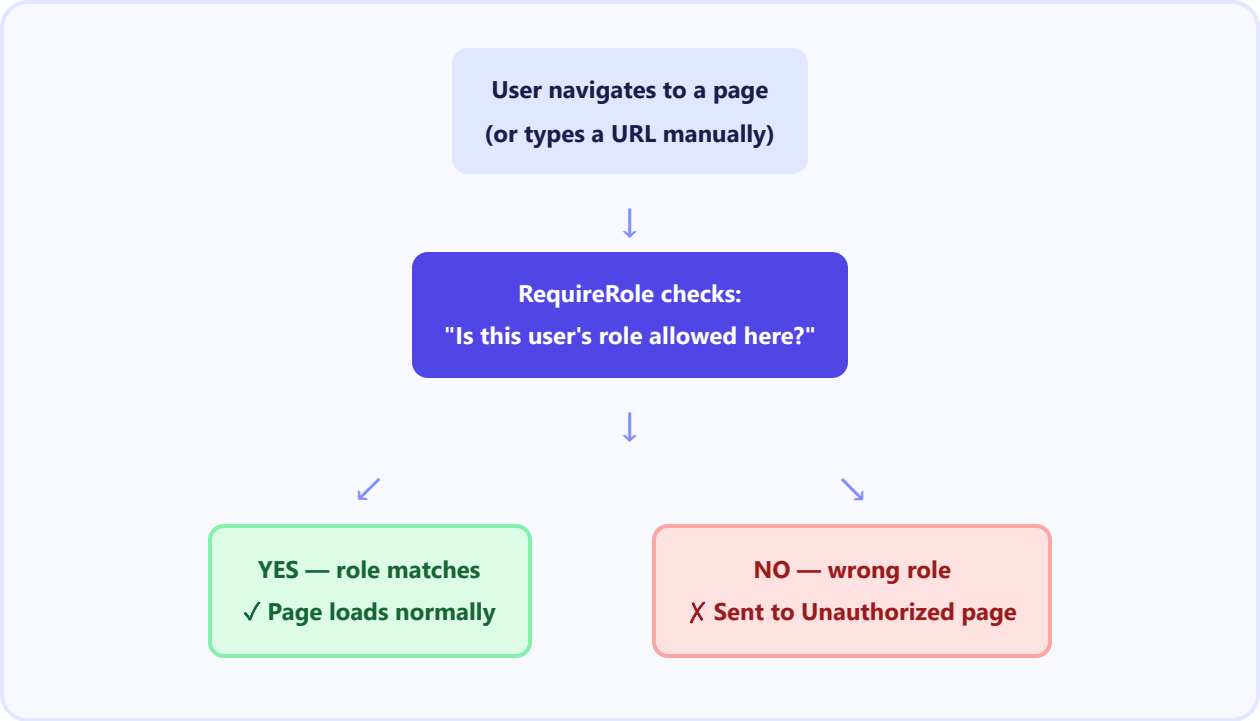
4. **The correct dashboard and menu items appear**
   The sidebar reads the role and shows only the relevant links. The wrong role's pages are never even rendered.

> **Say to your mentor:** "*When a user logs in, the app fetches their profile from Firestore which contains their role — doctor, patient, or pharmacist. This role is stored in a React Context so every component in the app can read it. The sidebar and the navigation then show only the menu items that belong to that role.*"

5 The Security Guard — How Pages Are Protected

ANALOGY — THINK OF IT LIKE A BOUNCER AT A CLUB

Before you enter any room, a bouncer checks your ID (your role). If your ID says you're allowed in, you pass. If not, you're turned away — even if you know the address. In MedFind this bouncer is a piece of code called **RequireRole**.



Example — What happens if a Patient tries to access the Prescription form?

! **Patient types `/dashboard/new-prescription` in the browser**

This is a Doctor-only page.

→ **RequireRole runs instantly**

It reads the role from Context: "patient". The allowed list for this page is: ["doctor"]. No match found.

✓ **Patient is redirected to Unauthorized page**

The prescription form is never shown. The data is also protected by Firebase rules — so even a direct database request would be blocked.

> **Say to your mentor:** *"Every protected route in the app is wrapped by a component called RequireRole. It checks the user's role from the global context before rendering any page. If the role doesn't match, the user is immediately redirected to an Unauthorized page — even if they manually type the URL. On top of that, Firebase security rules protect the database directly, so a clever user can't bypass the frontend and query the data directly either."*

6 Firebase — Your Ready-Made Backend

ANALOGY — THINK OF FIREBASE LIKE A FULLY-EQUIPPED KITCHEN

You didn't build the kitchen from scratch — you rented one that already has ovens, refrigerators, and a storage room. You just put your ingredients (data) in it and use the appliances (authentication, database). Firebase is Google's kitchen.

Firebase Authentication

- Manages email + password login
- Stores passwords securely (hashed)
- Issues a token when user logs in
- Tells the app: "User is logged in / out"
- Handles sign up, sign in, sign out

↔

Your React App

Firebase Firestore (Database)

- Stores all app data as documents
- Collections: profiles, prescriptions, medicines, inventory, orders
- Real-time: changes appear instantly
- Security rules: users can only read/write their own data

What data is stored in Firestore?

Collection (Table)	What it stores	Who uses it
profiles	User info: name, email, role, verified status	Everyone (on login)
prescriptions	All prescriptions: patient, doctor, medicines, expiry date	Doctor writes, Patient reads, Pharmacy reads
prescription_items	Each medicine in a prescription (dosage, frequency, duration)	Doctor writes, Patient reads
medicines	The global medicine database (name, strength, substance)	Doctor, Pharmacist, Patient (search)
inventory	Each pharmacy's stock: quantity, price per medicine	Pharmacist manages

Collection (Table)	What it stores	Who uses it
orders	Prescription orders sent to pharmacies	Patient creates, Pharmacist fulfills
connections	Doctor–patient relationships (approved or pending)	Doctor accepts, Patient sends

> **Say to your mentor:** "*Firebase gives me two things: Authentication for login, and Firestore as the database. Firestore is a NoSQL cloud database — instead of tables with rows like SQL, it stores documents in collections. I have collections for profiles, prescriptions, medicines, inventory, and so on. Firebase security rules ensure users can only access their own data.*"

7 How a Page Loads — The Full Journey

Let's trace exactly what happens when a **Doctor logs in and opens the New Prescription page**. This is the complete picture of your architecture in action.

1. **Doctor opens the browser and goes to the app URL**
   The browser downloads your React app. At this point, no one is logged in. The app shows the Login page.

2. **Doctor types email + password and clicks "Sign In"**
   The app sends the credentials to Firebase Authentication. Firebase checks them and says "OK".

3. **App fetches the doctor's profile from Firestore**
   Using the user ID from Firebase Auth, the app reads the profile document from the "profiles" collection. It gets: name, role = "doctor", verified = true.

4. **Profile is saved to the global notice board (Context)**
   Every component in the app can now read: "This is Dr. Ahmed. Role: doctor."

5. **Router redirects the doctor to /dashboard**
   The router reads the role and sends the doctor to their dashboard. The sidebar shows Doctor-specific menu items.

6. **Doctor clicks "New Prescription" in the sidebar**
   The router tries to load the New Prescription page. But first, RequireRole runs.

7. **RequireRole checks: "Is 'doctor' in the allowed list?"**
   Allowed list for this page: ["doctor"]. Role is "doctor". ✓ Pass.

8. **Page code is downloaded (lazy loading)**
   Because this page uses lazy loading, its JavaScript file only downloads now — not when the app first loaded. A spinner shows for a moment.

9. **Page fetches its data from Firestore**
   The New Prescription page needs: the doctor's patients (from "connections" collection) and the medicine list (from "medicines" collection). These are fetched from Firestore.

✓ **The form appears with the doctor's data pre-loaded**

Patient dropdown shows connected patients. Medicine dropdown shows the medicines database. The doctor can now write a prescription.

8 Key Terms — In Plain English

These are the words your mentor might use. Know what they mean — not how to spell them in code.

Term	Plain English Meaning
Component	A reusable piece of the UI. Like a LEGO brick. A button is a component. A card is a component. A page is made of many components.
Page	What shows when you go to a specific URL. Made of components. Example: the New Prescription page is one big component made of smaller form components.
Route	The rule that says: "When the URL is /dashboard/prescriptions, show the Prescriptions page." Router.jsx contains all the routes.
State	Data that can change. Example: the loading spinner is shown while loading = true and hidden when loading = false. State is what makes the UI react to things happening.
Context	A global notice board that every component can read. Instead of passing the user's role through every component manually, you put it in Context once and read it anywhere.
Props	Data passed from a parent component to a child. Like giving instructions when you ask someone to do a job: "Here is the list of patients — show them in a dropdown."
Hook	A special React function that lets a component do something — like remember data (useState), fetch data when the page loads (useEffect), or read from the global notice board (useContext).
Lazy Loading	Only download the code for a page when the user actually visits it. Keeps the app fast on first load.
Guard / RequireRole	A component that checks your role before letting you into a page. Like a bouncer with a guest list.

Term	Plain English Meaning
Firestore	Firebase's database. Stores data as "documents" inside "collections" (like files inside folders). Everything in your app — prescriptions, profiles, medicines — is a document.
Authentication	"Who are you?" — Verifying the user's identity via email and password login.
Authorization	"What are you allowed to do?" — Checking the user's role to decide which pages and data they can access.
NoSQL	A type of database that stores flexible documents instead of rigid rows and columns. Firestore is NoSQL.
Async / Await	When the app needs to wait for something (like Firebase to reply), it uses async/await. The rest of the app keeps working while it waits — it does not freeze.

9 Architecture Summary — One Page

Everything on one diagram. If your mentor asks "walk me through your architecture", point at this.

MedFind — Full Architecture Overview

🩺 **Doctor** | 🧑 **Patient** | 💊 **Pharmacist** | 🛡 **Admin**

↓ logs in via

Login Page — Email + Password → Firebase Authentication confirms identity

↓ identity confirmed → fetch profile from

🔥 **Firebase Firestore — profiles collection** — gets: name, role, verified status

↓ role stored in

React Context (Global Notice Board) — role available to every part of the app

↓ router reads role → sends user to

Dashboard Layout
Sidebar · Header · Main content area

+

RequireRole Guard
Checks role before every page loads

↓ correct role → shows

Doctor Pages
New Prescription
Patients · Pharmacies
Connections

Patient Pages
Prescriptions
Dose Schedule
My Doctors

Pharmacy Pages
Inventory · Medicines
Orders

Admin Pages
Users Manager

↓ pages read & write data from

🔥 **Firebase Firestore Collections**

profiles | prescriptions | prescription_items | medicines | inventory | orders | connections

> **Say to your mentor:** *"The user logs in through Firebase Authentication. The app then fetches their profile and role from Firestore and stores it in React Context. The router redirects them to their dashboard, and every page is wrapped in a role guard that checks the role before rendering. When a page loads, it fetches its specific data from Firestore collections — prescriptions, medicines, inventory, and so on."*